

Phase-Interference Energy and the Formal Structure of the PG1224 Prime Generation System

A Lean 4 Formalization of Prime = Energy 0 = Stable Fixed Point

Takuya Sogawa

2026-06-01

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

Version: v0.1.0

Date: 2026-06-01

DOI: 10.5281/zenodo.20483437

License: CC BY 4.0

Canonical version: English (paper-en.md / paper-en.pdf)

Japanese version: Companion translation (paper-ja.md / paper-ja.pdf)

Abstract

This research note reformulates the elementary number-theoretic structure of natural numbers using the operational concept of **phase-interference energy**. In this model, a prime number is treated as a stable fixed point that contains no non-trivial internal interference.

In this paper, the interference energy of a natural number n is defined as a measure of its non-trivial divisors, namely internal factors m satisfying $1 < m < n$ and $m \mid n$. A composite number contains at least one such non-trivial divisor, and therefore its interference energy becomes positive. Conversely, among natural numbers greater than or equal to 2, the absence of non-trivial divisors characterizes a zero-energy stable state.

By formalizing this intuition in Lean 4, the relationships among `Nat.Prime n`, `interferenceEnergy n = 0`, and `isStableFixedPoint n` can be treated not merely as metaphors, but as mechanically checkable logical propositions. Since 0 and 1 may also satisfy `interferenceEnergy = 0`, this note defines stable fixed points with the domain condition $2 \leq n$. This makes the claim “Prime = Energy 0 = Stable Fixed Point” mathematically safe over the full natural-number domain.

The note also introduces **PG1224**, a prime-candidate generation system based on residue classes modulo 12 and 24. Residue-class filtering is interpreted as a form of interference cancellation: trivial interference caused by divisibility by 2 or 3 is removed before the remaining candidates are connected to Lean’s primality predicate. In this way, candidate generation, formal verification, and stable-fixed-point semantics are integrated into a single specification-level model.

Keywords

Prime numbers; Interference energy; PG1224; Lean 4; Formal verification; Residue-class filtering; Stable fixed point; AIKernel

1. Background and Motivation

1.1 What is phase-interference energy?

In this research note, **phase-interference energy** is an operational concept for measuring the non-trivial decomposability of a natural number. It treats the existence of internal divisor structure as a form of interference.

In elementary number theory, a composite number is usually defined statically as a natural number having a divisor other than 1 and itself. This note views the same structure from a slightly different angle. When a natural number n has a non-trivial divisor m , the number n contains an internal periodic structure smaller than itself. Under the phase-model interpretation, such an internal period can be read as an interference component embedded within n .

Under this interpretation, a composite number is an excited state in which multiple periodic components overlap, while a prime number is a pure state without a non-trivial internal period. A prime can therefore be regarded as a point with zero interference energy, or equivalently as a stable fixed point that has no internal transition toward further divisor decomposition.

1.2 Why treat primes as energy-minimal points?

A prime number p has no divisors other than 1 and p . Equivalently, there is no divisor m such that $1 < m < p$ and $m \mid p$.

If interference energy is defined as the amount of non-trivial divisor structure, then this energy is zero for prime numbers. In contrast, a composite number has at least one non-trivial divisor, and therefore its interference energy becomes positive.

The correspondence can be summarized as follows.

Prime
= no non-trivial divisor
= zero interference energy
= stable fixed point under divisor-collapse semantics

There is, however, an important boundary condition. The natural numbers 0 and 1 also have no non-trivial divisors in the above sense. If one simply defines stability as $\text{interferenceEnergy } n = 0$, then 0 and 1 would incorrectly be included as stable prime-like states.

For this reason, this note explicitly includes the domain condition $2 \leq n$ whenever the equivalence with primality is stated. This is not a minor technicality; it is the condition that keeps the formalization sound over the natural numbers.

1.3 Residue-class filtering and cancellation

Residue-class filtering is a basic and powerful method for reducing prime candidates. In particular, multiples of 2 and multiples of 3 can be excluded before more expensive checks are performed.

Modulo 12, every prime greater than 3 lies in one of the following residue classes:

1, 5, 7, 11 mod 12

Similarly, modulo 24, every prime greater than 3 lies in one of the residue classes coprime to 24:

1, 5, 7, 11, 13, 17, 19, 23 mod 24

This note interprets residue-class filtering as **cancellation of trivial interference**. Interference arising from divisibility by 2 or 3 is removed first, thereby shrinking the candidate space in which zero-energy points may remain.

1.4 Why formalize the model in Lean 4?

The expression “Prime = Energy 0 = Stable Fixed Point” is intuitive, but intuition alone is not a mathematical proof.

Lean 4 is useful here for the following reasons:

1. It forces definitions such as `def` and `theorem` to have explicit meanings.

2. It makes boundary cases, especially 0 and 1, impossible to ignore.
3. It separates the metaphorical term “energy” from the logical structure of divisibility.
4. Proofs can be decomposed into small lemmas and mechanically checked.
5. It clarifies the responsibility boundary between PG1224 candidate generation and primality verification.

Accordingly, this note does not propose a new definition of prime numbers. It reconstructs the existing notion of primality in the vocabulary of energy minimization and stable fixed points, and places that reconstruction into a Lean-checkable form.

2. Definition and Semantics of Interference Energy

2.1 Non-trivial divisors

For a natural number n , a non-trivial divisor is a natural number m satisfying:

$$1 < m \wedge m < n \wedge m \mid n$$

This condition states that m is a genuine internal decomposition component of n .

In Lean-style notation, the predicate can be written as follows.

```
def isNontrivialDivisor (n m : Nat) : Prop :=
  1 < m ∧ m < n ∧ m ∣ n
```

2.2 interferenceEnergy

The value `interferenceEnergy n` is defined as the number of non-trivial divisors of n .

Operationally, this can be implemented by constructing a finite search space from 0 to n and counting the elements that satisfy the divisor condition.

```
def nontrivialDivisors (n : Nat) : List Nat :=
  (List.range (n + 1)).filter (fun m =>
    1 < m ∧ m < n ∧ n % m = 0)

noncomputable def interferenceEnergy (n : Nat) : Nat :=
  (nontrivialDivisors n).length
```

A concrete example makes the intuition immediate. The non-trivial divisors of 12 are {2, 3, 4, 6}, so `interferenceEnergy 12 = 4`. The number 13 has no non-trivial divisors, so `interferenceEnergy 13 = 0`.

```
12 -> {2, 3, 4, 6} -> energy = 4
13 -> {}           -> energy = 0
```

In an actual Lean implementation, additional lemmas are required to connect `%`, divisibility, `List.filter`, and decidability. The key point for the present specification is that interference energy is a natural-number-valued quantity and therefore never becomes negative.

2.3 State semantics

This definition allows the state of a natural number n to be interpreted as follows.

State	Number-theoretic meaning	Interference-model meaning
<code>interferenceEnergy n = 0</code>	no non-trivial divisor exists	no internal interference
<code>interferenceEnergy n > 0</code>	a non-trivial divisor exists	internal interference is excited

However, `interferenceEnergy n = 0` alone includes both 0 and 1. Therefore, the equivalence with primality must be stated together with $2 \leq n$.

2.4 Stable fixed points

In the phase-generation interpretation, a stable fixed point is a state that cannot collapse into a non-trivial internal component.

The corresponding Lean-style definition is:

```
def isStableFixedPoint (n : Nat) : Prop :=  
  2 ≤ n ∧ interferenceEnergy n = 0
```

The condition $2 \leq n$ excludes the boundary states 0 and 1. With this definition, stable fixed points become candidates for exact equivalence with prime numbers.

3. Construction of PG1224

3.1 Basic idea of PG1224

PG1224¹ is a prime-candidate generation system that uses residue classes modulo 12 and 24.

The name is used here to mean a Prime Generator based on the residue-class structure of bases 12 and 24.

The role of PG1224 is not to test all natural numbers uniformly, but to remove candidates that are obviously composite and to shrink the candidate space in which zero-energy points may remain.

3.2 Candidate set

The PG1224 candidate set is:

```
{2, 3}  
∪ { n | n mod 12 ∈ {1, 5, 7, 11} }  
∪ { n | n mod 24 ∈ {1, 5, 7, 11, 13, 17, 19, 23} }
```

In Lean-style notation:

```
def candidate12 (n : Nat) : Prop :=  
  n % 12 = 1 ∨ n % 12 = 5 ∨ n % 12 = 7 ∨ n % 12 = 11  
  
def candidate24 (n : Nat) : Prop :=  
  n % 24 = 1 ∨ n % 24 = 5 ∨ n % 24 = 7 ∨ n % 24 = 11 ∨  
  n % 24 = 13 ∨ n % 24 = 17 ∨ n % 24 = 19 ∨ n % 24 = 23  
  
def candidate_PG1224 (n : Nat) : Prop :=  
  n = 2 ∨ n = 3 ∨ candidate12 n ∨ candidate24 n
```

This candidate set does not directly generate only primes. It is a candidate filter that keeps residue classes in which primes may occur.

3.3 Generation predicate

PG1224 treats n as a generated prime only when candidate membership and primality are both established.

```
def generatesImpl_PG1224 (n : Nat) : Prop :=  
  candidate_PG1224 n ∧ Nat.Prime n
```

Because this definition includes `Nat.Prime n`, `generatesImpl_PG1224` is not a stand-alone primality test. It is a **certified generation predicate** that combines a candidate filter with a proof of primality.

This responsibility boundary is important. PG1224 performs candidate-space cancellation, while final primality is guaranteed by Lean's `Nat.Prime` predicate or by an equivalent proof.

3.4 Soundness, completeness, and minimality

The formal evaluation of PG1224 is organized around three properties.

¹PG1224 is a convenient name for a Prime Generator that combines residue-class filtering modulo 12 and 24.

Soundness Every element generated by PG1224 is prime, and therefore a zero-energy stable fixed point.

```
theorem pg1224_sound (n : Nat) :
  generatesImpl_PG1224 n → Nat.Prime n := by
  intro h
  exact h.right
```

This follows immediately from the definition of `generatesImpl_PG1224`.

Completeness Every prime is included in the PG1224 candidate set.

```
theorem prime_implies_candidate_PG1224 (n : Nat) :
  Nat.Prime n → candidate_PG1224 n := by
  -- Split the prime cases into n = 2, n = 3, and n > 3.
  -- If n > 3, primality implies that n is divisible by neither 2 nor 3.
  -- Therefore gcd n 6 = 1, so n falls into the residue classes coprime to 6,
  -- which are represented by the PG1224 residues modulo 12 and 24.
  sorry
```

The proof sketch is as follows. First, handle the special cases $n = 2$ and $n = 3$, which are explicitly included in the candidate set. For $n > 3$, primality implies $\neg 2 \mid n$ and $\neg 3 \mid n$. Hence n is coprime to 6. Therefore n falls into residue classes modulo 12 and 24 that are unaffected by the trivial interference caused by 2 and 3, and is included in the PG1224 candidate set.

This gives:

```
theorem pg1224_complete (n : Nat) :
  Nat.Prime n → generatesImpl_PG1224 n := by
  intro hp
  exact And.intro (prime_implies_candidate_PG1224 n hp) hp
```

Minimality In this note, minimality does not mean global optimality among all possible prime generation methods. It refers only to local redundancy reduction: residue-class filtering modulo 12 and 24 removes trivial interference caused by divisibility by 2 and 3 before later verification steps.

A stronger optimality claim would require a separately defined candidate space, cost function, and admissible residue basis.

4. Structural Theorems of Interference Energy

4.1 Non-negativity

Interference energy is a natural number, so it is always non-negative.

```
theorem interferenceEnergy_nonneg (n : Nat) :
  0 ≤ interferenceEnergy n := by
  exact Nat.zero_le _
```

Although simple, this theorem guarantees that the energy does not take non-physical negative values.

4.2 Composite numbers raise energy

If n is composite, then it has a non-trivial divisor. Therefore its interference energy is positive.

```
theorem interferenceEnergy_pos_of_composite (n : Nat) :
  Composite n → 0 < interferenceEnergy n := by
  -- A composite number has a non-trivial divisor.
  -- Therefore the filtered divisor list is non-empty.
  sorry
```

Here `Composite n` can be introduced as an auxiliary predicate:

```
def Composite (n : Nat) : Prop :=
  ∃ m, 1 < m ∧ m < n ∧ m ∣ n
```

This theorem formalizes the statement that every composite number contains at least one interference component.

4.3 Divisor monotonicity

Along a divisor chain, internal structure is preserved or increases.

Intuitively, if $d \mid n$ and $2 \leq d < n$, then non-trivial divisor structure inside d is embedded into the internal structure of n . Under appropriate conditions, one expects the following monotonicity property.

```
theorem interferenceEnergy_divisor_monotone (d n : Nat) :  
  2 ≤ d → d < n → d ∣ n →  
  interferenceEnergy d ≤ interferenceEnergy n := by  
  -- Non-trivial divisors of d are mapped into non-trivial divisors of n.  
  sorry
```

This theorem expresses the idea that a more complex multiple structure does not reduce the potential amount of internal interference.

4.4 Cancellation by a non-trivial divisor

If at least one non-trivial divisor exists, then interference energy rises to at least 1.

```
theorem interferenceEnergy_ge_one_of_has_nontrivial_divisor (n : Nat) :  
  (∃ m, 1 < m ∧ m < n ∧ m ∣ n) →  
  1 ≤ interferenceEnergy n := by  
  -- The divisor m appears in the filtered list.  
  -- Therefore the list length is at least one.  
  sorry
```

This theorem is important for the cancellation semantics of PG1224. Detecting even one non-trivial interference component is sufficient to exclude a candidate from the zero-energy set.

5. Prime = Energy 0 = Stable Fixed Point

5.1 Energy zero and absence of non-trivial divisors

First, zero interference energy is equivalent to the absence of non-trivial divisors.

```
theorem energy_zero_iff_no_nontrivial_divisors (n : Nat) :  
  interferenceEnergy n = 0 ↔  
  ¬ ∃ m, 1 < m ∧ m < n ∧ m ∣ n := by  
  -- The filtered list has length zero iff it contains no element.  
  sorry
```

This lemma is the central bridge between list length and existential divisibility.

5.2 Prime iff energy zero under $n \geq 2$

To exclude 0 and 1, we assume $2 \leq n$.

```
theorem prime_iff_energy_zero_of_two_le (n : Nat) (hn : 2 ≤ n) :  
  Nat.Prime n ↔ interferenceEnergy n = 0 := by  
  -- Forward direction:  
  -- a prime has no non-trivial divisor.  
  -- Backward direction:  
  -- if  $n \geq 2$  and no non-trivial divisor exists,  $n$  is prime.  
  sorry
```

This theorem makes the statement “prime = zero interference energy” precise under the required domain condition.

5.3 Prime iff stable fixed point

Because `isStableFixedPoint` includes $2 \leq n$, the following theorem can be stated over the full natural-number domain.

```
theorem prime_iff_stable_fixed_point (n : Nat) :  
  Nat.Prime n ↔ isStableFixedPoint n := by  
  constructor  
  · intro hp  
    exact And.intro  
      (Nat.Prime.two_le hp)
```

```

      ((prime_iff_energy_zero_of_two_le n (Nat.Prime.two_le hp)).mp hp)
    · intro hs
      exact (prime_iff_energy_zero_of_two_le n hs.left).mpr hs.right

```

This safely formalizes the statement “Prime = Energy 0 = Stable Fixed Point” without incorrectly including 0 and 1.

5.4 Integration with PG1224

If PG1224 completeness is available, the following equivalence can be constructed.

```

theorem prime_iff_pg1224_generated (n : Nat) :
  Nat.Prime n ↔ generatesImpl_PG1224 n := by
  constructor
  · exact pg1224_complete n
  · exact pg1224_sound n

```

Combining PG1224 generation, primality, stable fixed points, and zero interference energy gives the following unified theorem.

```

theorem pg1224_unified_equivalence (n : Nat) :
  Nat.Prime n ↔
    generatesImpl_PG1224 n ∧
    isStableFixedPoint n ∧
    interferenceEnergy n = 0 := by
  constructor
  · intro hp
    exact And.intro
      ((prime_iff_pg1224_generated n).mp hp)
      (And.intro
        ((prime_iff_stable_fixed_point n).mp hp)
        ((prime_iff_energy_zero_of_two_le n (Nat.Prime.two_le hp)).mp hp))
  · intro h
    exact (prime_iff_pg1224_generated n).mpr h.left

```

In Lean, chained expressions such as $A \leftrightarrow B \leftrightarrow C \leftrightarrow D$ may be affected by associativity. For public specifications, it is clearer to split the proof into lemmas and then collect the result as a product, as shown above.

6. Semantics of the Proof Structure

6.1 Soundness

Soundness means that every object generated by the system is actually prime.

For PG1224, this follows directly because the generation predicate is defined as $\text{candidate_PG1224 } n \wedge \text{Nat.Prime } n$.

This may look trivial, but it is important as a design boundary. PG1224 does not conflate the candidate-generating layer with the primality-verifying layer. In AIKernel terms, the candidate generator is a Provider, while primality verification belongs to a Verifier or Governance layer.

6.2 Completeness

Completeness means that every prime lies in the PG1224 candidate space.

Every prime greater than 3 is divisible by neither 2 nor 3. Therefore, modulo 12 or 24, it belongs to a residue class coprime to 2 and 3. This is the basis of PG1224 completeness.

6.3 Stable fixed point

A stable fixed point is a point that has no non-trivial internal collapse through divisors.

The equation $\text{interferenceEnergy } n = 0$ means that no non-trivial divisor exists. Together with $2 \leq n$, this is equivalent to $\text{Nat.Prime } n$.

Thus, a stable fixed point is a natural-number state that cannot be decomposed non-trivially. It is the same object as a prime number in elementary number theory, but expressed through dynamic and generative-system semantics.

7. Possible Applications

7.1 Controlling fluctuation in LLM inference

The central idea of this note is broader than prime numbers.

Interference energy can be abstracted as a quantity that measures non-trivial contradictions, branches, or collapse factors inside a state. Applied to an LLM inference trajectory, semantic contradiction, goal drift, unstable tool selection, or excessive recursive correction can be treated as forms of interference energy.

In the context of AIKernel Trajectory Governance, a system may observe whether inference is converging toward a stable fixed point or whether interference energy is rising. If the latter occurs, the governance layer may fail closed.

7.2 Significance as a topological number-theoretic model

This model treats primes not only as a static set, but also as stable points remaining after interference cancellation.

This does not conflict with the traditional definition of primes as indecomposable numbers. Rather, it translates that definition into the vocabulary of phase structure, candidate generation, cancellation, energy minimization, and fixed points.

Through this translation, residue-class filtering, sieves, generators, energy minimization, and fixed-point semantics can be placed into a common conceptual frame.

7.3 OS-level application to generative systems

In an AI orchestration OS such as AIKernel, the runtime must decide whether an input task can be executed safely.

The PG1224 structure can be read as a small formal model of such a decision process:

1. Exclude trivially dangerous or inappropriate candidates using a preprocessing step analogous to residue-class filtering.
2. Evaluate interference energy on the remaining candidates.
3. Pass only zero-energy stable candidates to the execution layer.
4. Route positive-energy candidates to confirmation, quarantine, or fail-closed behavior.

In this sense, PG1224 is not merely a number-theoretic candidate generator. It is also a compact formal analogy for a governance gateway in AIKernel.

8. Non-Claims

This note does not claim the following:

1. It does not replace the standard definition of prime numbers.
2. It does not claim that PG1224 is computationally superior to existing sieve methods.
3. `generatesImpl_PG1224` is a certified generation predicate containing `Nat.Prime`; it is not a stand-alone primality tester.
4. Residue-class filtering alone does not remove all composite numbers.
5. This note does not prove any open problem in analytic number theory.
6. The term “energy” is not a physical quantity; it is an operational and semantic name for divisor structure.

These non-claims delimit the intended contribution of this note.

9. Conclusion

This research note defined the non-trivial divisor structure of natural numbers as interference energy and reconstructed prime numbers as zero-energy stable fixed points.

The key technical point is that $\text{interferenceEnergy } n = 0$ alone does not exclude 0 and 1. By including $2 \leq n$ in the definition of `isStableFixedPoint`, the equivalence $\text{Nat.Prime } n \leftrightarrow \text{isStableFixedPoint } n$ can be stated safely over the entire natural-number domain.

The note also positioned PG1224, based on residue-class filtering modulo 12 and 24, as a candidate generation system grounded in interference cancellation. PG1224 separates candidate generation from primality proof and provides a formal framework for connecting soundness, completeness, and stable-fixed-point semantics in Lean 4.

The central contribution is not a new concept of primality. It is a reconstruction of the existing notion of primality in the vocabulary of energy minimization, interference cancellation, stable fixed points, and formal verification, with a path toward AIKernel-style governance models.

References

- The mathlib Community. (2020). The Lean mathematical library. Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, 367-381. <https://doi.org/10.1145/3372885.3373824>
- de Moura, L., Kong, S., Avigad, J., van Doorn, F., & von Raumer, J. (2015). The Lean Theorem Prover. Automated Deduction - CADE-25, Lecture Notes in Computer Science, 9195, 378-388. Springer. https://doi.org/10.1007/978-3-319-21401-6_26
- Lean FRO. (2026). Lean 4 Manual. Lean Focused Research Organization.
- Sogawa, T. (2026). AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference. Zenodo.
- Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo.